Project Report: Expense Manager

1. Cover Page

Field		Detail
Project Title	-	Expense Manager System
Submitted by	-	Vaidehi Upadhyay
Registration Number	-	25BCY10027
Submission Date	-	November 25, 2025
Submitted to	-	VITyarthi

2. Introduction

This project details the development of a **Simple Expense Manager**, a command-line interface (CLI) application built using Python. The application is designed for individual users to track daily expenditures, categorize them, and monitor spending against a predefined monthly budget. The system prioritizes simplicity and fast data entry, using local file storage for persistence.

- **Project Overview:** The Simple Expense Manager is a monolithic Python application that performs expense tracking and basic financial reporting (monthly summary) directly in the terminal.

- **Motivation:** The motivation was to create a basic, functional tool to understand core programming concepts like file I/O, data aggregation, string manipulation (date parsing), and simple application menus. It addresses the common user need for quick, accessible expense logging without the complexity of a full-scale accounting application.
- **Scope:** The system's scope is limited to three primary functions: adding expenses, setting a single monthly budget, and generating a summary for the current calendar month. It uses local text files (`expenses.txt` and `budget.txt`) for data persistence and does not include advanced features like user accounts, database integration, or historical reporting across years.

3. Problem Statement

Clearly articulate the specific problem, challenge, or need that your project addresses.

> **The Problem:** Individuals often rely on manual methods (notes or complex spreadsheets) to track daily expenses, making it tedious to quickly log transactions and instantly check spending against a monthly budget, especially without a dedicated application. This often leads to inconsistent tracking and potential overspending.
>
> **Proposed Solution:** This project develops a centralized, text-based command-line utility that allows users to quickly log expenses with minimal friction. It automates the calculation of category totals and provides an immediate summary of monthly spending versus the allocated budget, simplifying personal finance monitoring.

4. Functional Requirements

Functional requirements define what the system *must do*.

- **Expense Management:**
  - The system shall allow a user to input an expense, including amount, category, description, and date (YYYY-MM-DD).
  - The system shall persist all expense records to a local file (`expenses.txt`).
- **Budget Management:**
  - The system shall allow the user to set and save a single numerical monthly budget to a local file (`budget.txt`).
- **Reporting & Summary:**
  - The system shall calculate and display the total amount spent for the current calendar month.
  - The system shall calculate and display category-wise totals for the current month.
  - The system shall calculate and display the remaining budget for the current month (Budget - Total Spent).

- **Core Interface:**
    - The system shall display a main menu providing options for Add Expense, Monthly Summary, Set Budget, and Exit.

5. Non-functional Requirements

Non-functional requirements describe how the system *behaves* or *is*.

- **Simplicity & Usability:**
    - The user interface shall be purely text-based (CLI) for maximum simplicity.
    - Data input shall require minimal user steps and no complex formatting beyond the YYYY-MM-DD date standard.
- **Reliability & Persistence:**
    - The system shall successfully load all previous expenses and the current budget upon startup.
    - The system shall handle the absence of existing data files by creating new empty files or defaulting to zero budget.
- **Maintainability:**
    - The codebase is modular, separating file I/O logic from core application logic.
- **Performance:**
    - The system shall respond to all user inputs (Add, Summarize, Set Budget) instantaneously, as data processing involves minimal local file reading and simple arithmetic operations.

6. System Architecture

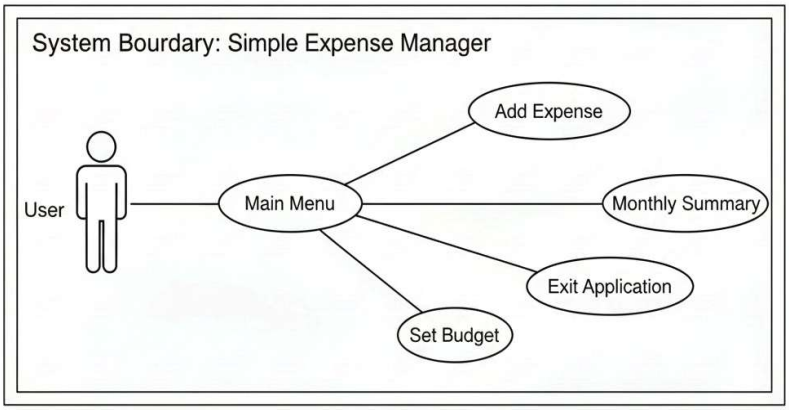
The Simple Expense Manager follows a straightforward **Monolithic Architecture** divided into two main logical components: the Application Logic and the Persistence Layer.

- **Architecture Pattern:** Monolithic Application (CLI).
- **Components:**
    - **Application Layer (Python Logic):** Contains all functions (`add_expense`, `monthly_summary`, etc.) responsible for user interaction, data processing, and business logic (e.g., aggregating monthly totals).
    - **Persistence Layer (File I/O):** Contains functions (`load_expenses`, `save_expenses`, etc.) responsible for interacting directly with the local file system.
- **Data Flow:** User input is processed by the Application Layer, which then reads/writes data to the Persistence Layer (local text files). Processed data (summaries) are displayed back to the user via the CLI.

7. Design Diagrams

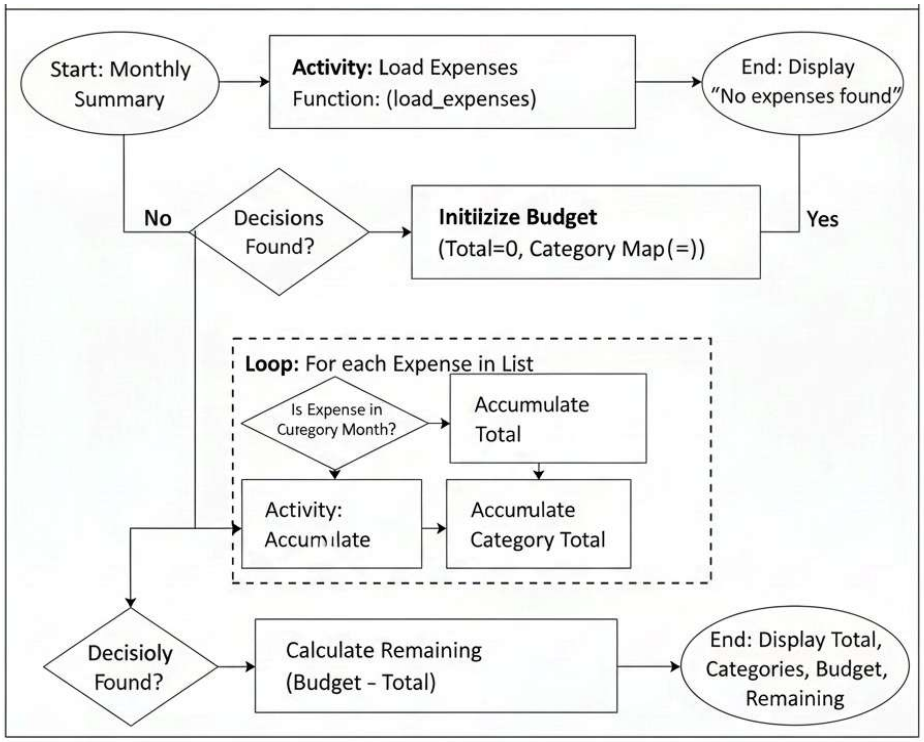
7.1 Use Case Diagram

Illustrates the user interactions with the core features of the system.



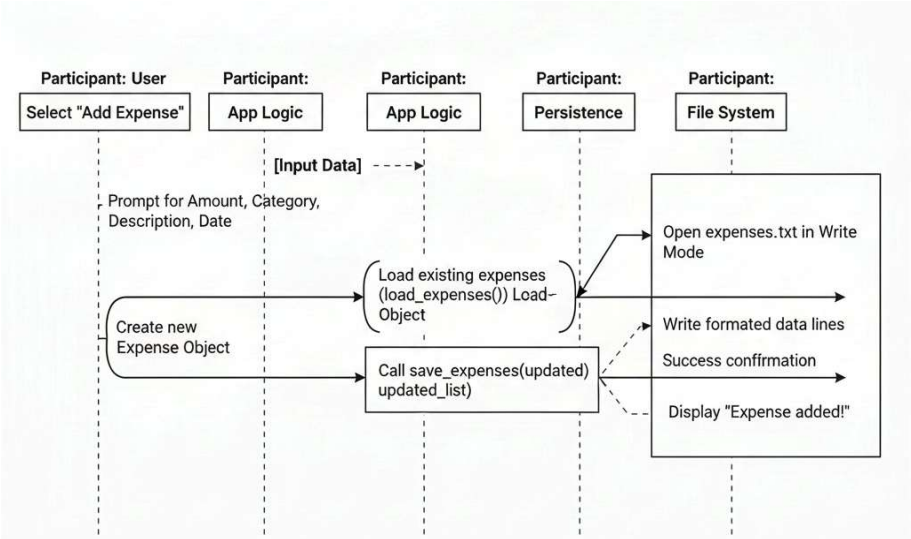
7.2 Workflow Diagram

Shows the sequence of steps and decisions necessary to complete the primary process of generating the monthly summary.



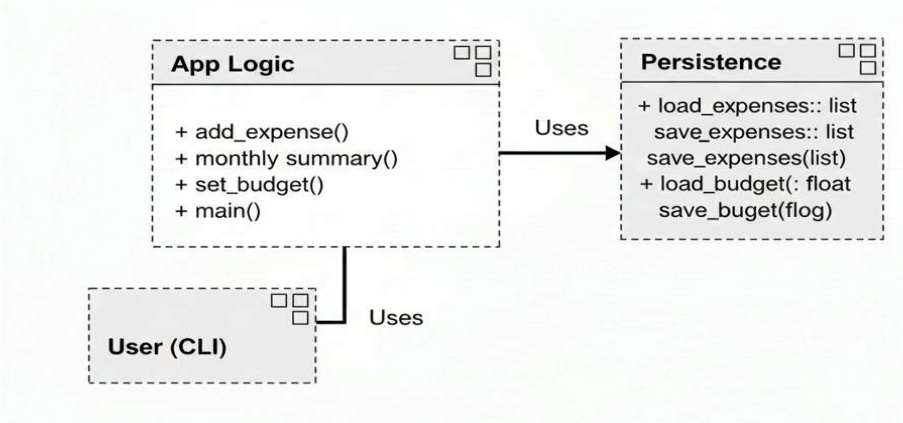
7.3 Sequence Diagram

Illustrates the order of messages and function calls required when the user initiates a key action, such as adding a new expense.



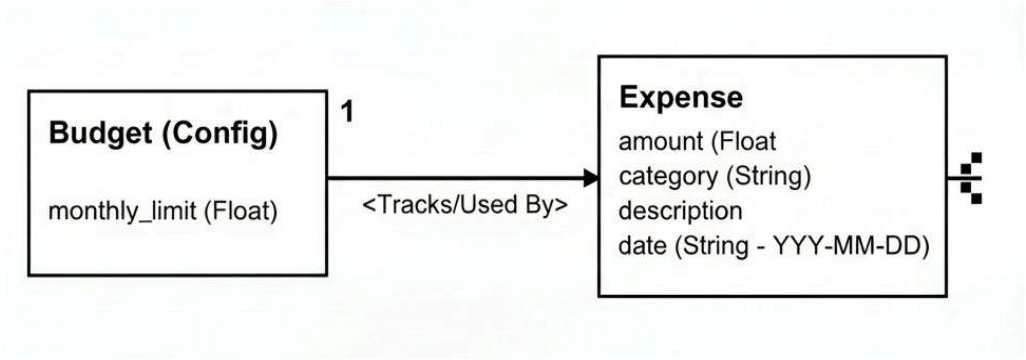
7.4 Class/Component Diagram

A structural diagram showing the main logical components of the Python application.



7.5 ER Diagram (Entity-Relationship Diagram)

Since the system uses local file storage acting as a simple data layer, a conceptual ERD can be provided for the primary entity: Expense.



8. Design Decisions & Rationale

Decision	Rationale	Alternatives Considered
Programming Language (Python)	Chosen for its simplicity, speed of development, and strong standard library support for file handling and date manipulation.	Java (Overly verbose for a simple CLI), Shell Scripting (Too limited for data aggregation).
Data Storage (Local Text Files)	Selected for the "Basic" nature of the project. It required no external dependencies or database configuration, fulfilling the requirement for simple data persistence.	SQLite (Would require database setup and SQL knowledge), JSON files (More complex parsing logic than pipe-delimited text).
Input Validation	Minimal input validation (relying on `float()` casting) to keep the code concise and focused on core logic, suitable for a basic learning project.	Implementing robust try-except blocks for data type and format checks (deferred to future enhancements).

Date Handling	Used the built-in `datetime` library for reliable current month extraction and expense filtering.	Manual string manipulation (less reliable and prone to bugs).

9. Implementation Details

Detail the core technical aspects and technologies used in the development.

- **Technology Stack:** Python 3.x, Standard Library (`datetime`).
- **Key Data Structures/Algorithms:**
  - **Data Structure:** Expenses are stored as a list of dictionaries in memory, where each dictionary represents one expense record.
  - **Persistence Format:** Data is stored in a pipe-delimited string format (`amount|category|description|date`) within the `expenses.txt` file.
  - **Algorithm:** The `monthly_summary` function implements a basic aggregation algorithm: iterating over all loaded expenses, filtering them by the current year and month, and accumulating the `total` and `category_total` sums.
- **Code Structure:** The application uses a modular procedural structure:
  - **I/O Functions:** `load_expenses`, `save_expenses`, `load_budget`, `save_budget`.
  - **Logic Functions:** `add_expense`, `monthly_summary`, `set_budget`.
  - **Main Loop:** `main()` function handles the CLI menu and routing.

10. Screenshots / Results

Since this is a CLI application, the results are represented by successful terminal interactions.

- **Interface Snapshots :**

  Main Menu Display: Shows the four primary numbered options.

```python
def main():
    while True:
        print("===== EXPENSE MANAGER =====")
        print("1. Add Expense")
        print("2. Monthly Summary")
        print("3. Set Budget")
        print("4. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            add_expense()
        elif choice == "2":
            monthly_summary()
        elif choice == "3":
            set_budget()
        elif choice == "4":
            print("Bye!")
            break
        else:
            print("Invalid choice\n")


main()
```

  Expense Addition: Demonstrates the user prompts for amount, category, description,

```python
def add_expense():
    print("\n--- Add Expense ---")
    amount = float(input("Enter amount: "))
    category = input("Enter category: ")
    desc = input("Enter description: ")
    date = input("Enter date (YYYY-MM-DD): ")

    expenses = load_expenses()

    expense = {
        "amount": amount,
        "category": category,
        "description": desc,
        "date": date
    }

    expenses.append(expense)
    save_expenses(expenses)

    print("Expense added!\n")


def monthly_summary():
    print("\n--- Monthly Summary ---")

    import datetime
    today = datetime.date.today()
    month = today.month
    year = today.year

    expenses = load_expenses()

    if len(expenses) == 0:
        print("No expenses found.\n")
        return
```

```python
    total = 0
    category_total = {}

    for e in expenses:
        y, m, d = map(int, e["date"].split("-"))
        if y == year and m == month:
            total += e["amount"]

            cat = e["category"]
            if cat not in category_total:
                category_total[cat] = 0
            category_total[cat] += e["amount"]

    budget = load_budget()
    remaining = budget - total

    print("\nTotal spent this month:", total)
    print("Category-wise totals:")
    for c, v in category_total.items():
        print(f"  {c}: {v}")

    print("\nBudget:", budget)
    print("Remaining:", remaining)
    print()


def set_budget():
    print("\n--- Set Budget ---")
    b = float(input("Enter monthly budget: "))
    save_budget(b)
    print("Budget updated!\n")
```

- **Example Output (Monthly Summary):**
    - The summary screen clearly outputs the total spent, a breakdown of category totals, the monthly budget, and the calculated remaining funds.

```
===== EXPENSE MANAGER =====
1. Add Expense
2. Monthly Summary
3. Set Budget
4. Exit
Enter choice: 3

--- Set Budget ---
Enter monthly budget: 40000
Budget updated!

===== EXPENSE MANAGER =====
1. Add Expense
2. Monthly Summary
3. Set Budget
4. Exit
Enter choice: 1

--- Add Expense ---
Enter amount: 500
Enter category: Food
Enter description: Outing to bhopal
Enter date (YYYY-MM-DD): 2025-11-23
Expense added!

===== EXPENSE MANAGER =====
1. Add Expense
2. Monthly Summary
3. Set Budget
4. Exit
Enter choice: 1

--- Add Expense ---
Enter amount: 3000
Enter category: Shopping
Enter description: Birthday Shopping
Enter date (YYYY-MM-DD): 2025-11-25
Expense added!
```

```
===== EXPENSE MANAGER =====
1. Add Expense
2. Monthly Summary
3. Set Budget
4. Exit
Enter choice: 2

--- Monthly Summary ---

Total spent this month: 4000.0
Category-wise totals:
  Food: 1000.0
  Shopping: 3000.0

Budget: 40000.0
Remaining: 36000.0

===== EXPENSE MANAGER =====
1. Add Expense
2. Monthly Summary
3. Set Budget
4. Exit
Enter choice: 4
Bye!
```

11. Testing Approach

Testing was conducted manually through the Command Line Interface due to the small scale and nature of the project.

- **Testing Types:**
    - **Unit Testing (Manual):** Verifying individual function integrity (e.g., ensuring `load_budget()` correctly reads a saved value or defaults to 0).
    - **Integration Testing:** Ensuring the save/load functions work seamlessly with the logic (e.g., adding an expense, exiting, restarting the program, and verifying the new expense is present in the summary).
    - **Scenario Testing:** Testing edge cases like having zero expenses, having a zero budget, and entering expenses across different months to ensure correct monthly filtering.
- **Tools Used:** Standard Python interpreter and the operating system's command-line terminal.

- **Test Cases (Example):**
  - **Test Case 1 (Persistence):** Add an expense and immediately check the contents of `expenses.txt`. Expected Result: A new, correctly formatted, pipe-delimited line is present.
  - **Test Case 2 (Monthly Filter):** Enter an expense dated for the previous month and run the Monthly Summary. Expected Result: The expense is *excluded* from the current month's total.
  - **Test Case 3 (Budget Status):** Set Budget to 500. Add Expenses totaling 600. Check Summary. Expected Result: Remaining budget shows -100 (overspent).

12. Challenges Faced

Discuss the significant technical or logistical difficulties encountered during the project and how you overcame them.

- **Challenge 1: Robust File Handling:** Initially, the code did not handle the `FileNotFoundError` when first running the application.
  - **Resolution:** Implemented explicit `try...except` blocks in the `load_expenses()` and `load_budget()` functions to gracefully handle file absence, returning an empty list or a default value of `0.0`, respectively.
- **Challenge 2: Date Comparison Logic:** Ensuring the summary correctly filtered expenses by month and year. Simple string comparison was unreliable for the `YYYY-MM-DD` format across year boundaries.
  - **Resolution:** Used the `import datetime` module and the `map(int, e["date"].split("-"))` logic to reliably parse the date components (year, month) as integers, allowing for accurate comparison with the current system date.

13. Learnings & Key Takeaways

Reflect on what you learned during the project development process.

- **Technical Learnings:** Deepened understanding of fundamental Python features, including local file I/O (`with open`), exception handling (`try...except`), and practical use of the `datetime` module for filtering time-series data.
- **Process Learnings:** Learned the importance of structured data storage, even when using plain text files, by adopting a consistent delimiter (`|`) to ensure reliable parsing. Also learned basic procedural programming design (separating I/O from logic).
- **Key Takeaway:** Even a simple application requires meticulous attention to data loading, saving, and format consistency to ensure data integrity and reliable reporting.

14. Future Enhancements

Outline potential features or improvements that could be added to the project in the future.

- **Short-Term:**
  - **Improved Persistence:** Migrate storage from plain text files to a dedicated, lightweight database like SQLite for more robust querying (e.g., fetching data from any specific month).
  - **Input Validation:** Implement stricter error checking for expense amount (must be positive number) and category (cannot be empty string).
- **Long-Term:**
  - **Advanced Reporting:** Add a function for annual summaries and trending reports, and the ability to filter expenses by arbitrary date ranges.
  - **GUI Development:** Convert the CLI into a desktop application using libraries like Tkinter or a web application using Flask/Django for better usability.

15. References

List all sources, libraries, documentation, and external articles that were used in the project development.

- Python 3 Documentation, Official Website.
- Standard Python Library Documentation (specifically `datetime` module).
- Basic file I/O tutorials for Python.